

PRIVACY WITHIN FEDERATED LEARNING SYSTEMS



Dr. Brian Ricks – Faculty Advisor - UTD

**Roland Varriale (Argonne National
Laboratory (ANL)) – Technical Director**

Dhyana Sai Reddy Dalli (DXD210031)

Lakshmi Saiesha Bonthala (LXB210017)

Neel Pareshbhai Patel (NPP200000)

Nikhil Satish Joshi (NSJ210001)

Shrey Saini (SXS190379)

Srinivas Panchangam (VXP210036)



AGENDA

Recap & Introduction

Federated Learning Methods

Privacy Risk & Privacy Protection

Dataset Security

Defense Mechanism

Results

Future Work

References

Recap

+



- Federated Learning: A machine learning approach for collaborative model training without directly sharing private data
- Working: Local models trained on each party's private data and sent to a central server for aggregation
- Privacy Leakage: Model inversion, membership inference, and model stealing attacks can compromise privacy
- Training Phase vs Inference Phase: Local models trained during the training phase, and the global model used for predictions in the inference phase
- Types of Attacks: Data poisoning, model inversion, membership inference, and model stealing attacks

Introduction

- Vulnerabilities of federated learning systems to attacks that compromise the integrity of the dataset used for training
- Focus of analyzing the dataset integrity and implementing security measures to prevent attacks.
- Identification of the most effective solution for preventing attacks, and the proposal of areas for future improvement and development.



FL Methods

Framework: TensorFlow Federated

Datasets: EMNIST <https://yann.lecun.com/exdb/mnist/>

Federated EMNIST - `tff.simulation.datasets.emnist`

Domain: Image Classification

Dataset: classes, (0-9 digits).



Privacy Risk in FL



- Data poisoning is a major privacy concern for FL, as malicious clients can intentionally change data features or labels to degrade the final learning model's quality.
- Label-flipping is a popular data poisoning attack in FL that flips or modifies labels in training data to cause misclassification or regression.
- The presence of even a small fraction of malicious participants in FL can dramatically affect the accuracy of the global model.
- Data poisoning is untraceable using conventional methods, but dimensionality reduction and clustering algorithms can detect it.
- Defense mechanisms are needed to ensure the robustness of FL against data poisoning attacks.

Dataset Security



- Data poisoning attacks can target various machine learning applications such as recommendation systems, differential privacy, reinforcement learning, and federated learning.
- Training-only attacks involve manipulating training data and labels and can be formulated as either from-scratch training or transfer learning.
- Backdoor attacks, also known as trojans, embed an exploit during training that is activated by a "trigger" during testing. These attacks can be model-agnostic or model-specific.
- Defense methods against dataset tampering can either detect when poisoning has occurred or resist poisons during the training process. Detection methods include identifying corrupted training instances or flagging corrupted models after training. Training-based defenses can use robust training routines or correct a corrupted model post-hoc to remove the effects of poisoning.

Defense Mechanism – Research Work 1



- UMAP is a powerful nonlinear dimensionality reduction method that is faster than PCA and t-SNE.
- UMAP is better than PCA in defense against data poisoning attacks due to its ability to preserve the local structure and geometry of the data.
- Automated defense strategies, such as PCA, KPCA and UMAP, can be used to locate and filter malicious updates in deep neural networks.
- The effectiveness of these techniques depends on the specific attack scenario and data characteristics, so careful consideration is necessary for choosing the appropriate technique.
- Reference: https://assets.researchsquare.com/files/rs-1984301/v1_covered.pdf?c=1668905976

Defense Mechanism – Research Work 2



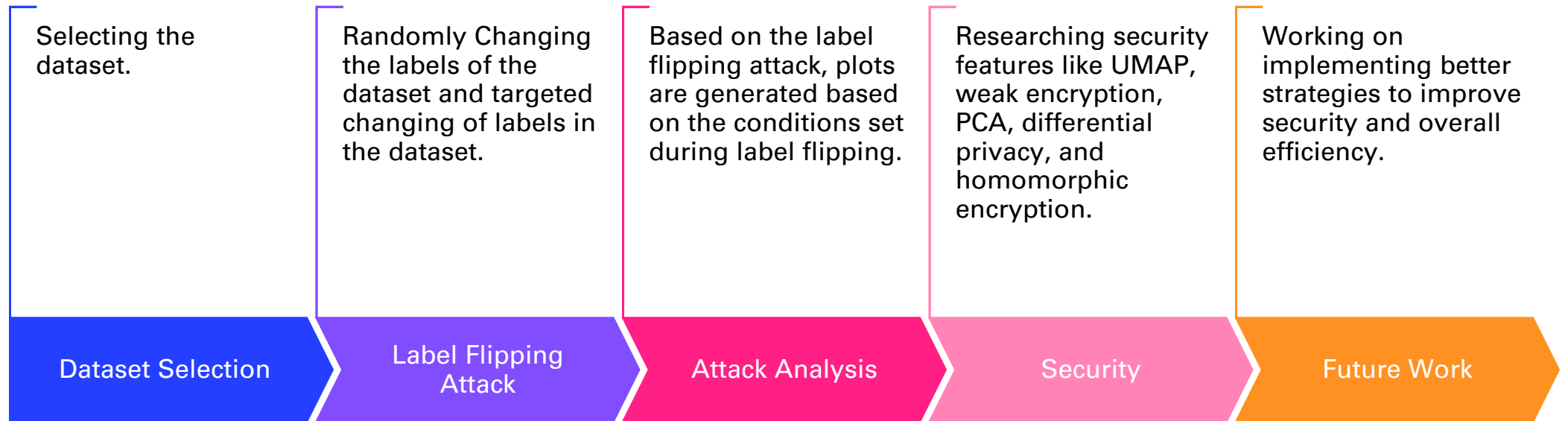
- Federated learning faces privacy issues that make it vulnerable.
- Various security protocols such as SMC and homomorphic encryption have computational overhead and limitations in terms of reliance on external participants.
- Differential privacy mechanism, such as the Laplace mechanism, can add noise to the intermediate parameters to reduce privacy risks, but it still has a drawback of privacy leakage if a malicious client is present.
- The proposed solution combines differential privacy and weak encryption to perturb privacy features, ensure non-informative communication and aggregation of parameters, and reduce the risk of internal and external attacks while maintaining training task accuracy.

Privacy Protection in FL



- Privacy preservation is a primary concern in federated learning as parties involved do not want to expose their private data to other parties.
- Several privacy-preserving techniques can be used, including Differential Privacy, Secure Multi-Party Computation (SMPC), Homomorphic Encryption, and Federated Averaging.
- Differential Privacy involves adding random noise to the training data to ensure no individual's data can be inferred.
- SMPC allows multiple parties to securely compute a function over their private data without revealing their inputs. Homomorphic Encryption enables computations on encrypted data, and Federated Averaging aggregates locally trained models to create a global model while ensuring individual party's data remains private.

Timeline



RESULTS



Label Flipping Setup

- Random Label Flipping

```
# Poison the second and third clients' training data by flipping the labels of a fraction of the data
poison_fraction = 0.5
# =====
# Label Flipping from 3 to 7 and vice versa with 2 malicious clients
# # Poison type 1: flip labels of a fraction of the data
poison_indices = np.random.choice(len(y2_train), int(len(y2_train) * poison_fraction), replace=False)
for idx in poison_indices:
    if y2_train[idx] == 1:
        y2_train[idx] = 7

poison_indices = np.random.choice(len(y3_train), int(len(y3_train) * poison_fraction), replace=False)
for idx in poison_indices:
    if y3_train[idx] == 3:
        y3_train[idx] = 8

# Retrain the models on the poisoned data
history1_poisoned = model1.fit(x1_train, y1_train, epochs=10, validation_data=(x1_test, y1_test))
history2_poisoned = model2.fit(x2_train, y2_train, epochs=10, validation_data=(x2_test, y2_test))
history3_poisoned = model3.fit(x3_train, y3_train, epochs=10, validation_data=(x3_test, y3_test))

# Evaluate the accuracy of the models on the test data again
print("Model 1 accuracy after poisoning:", model1.evaluate(x_test, y_test)[1])
print("Model 2 accuracy after poisoning:", model2.evaluate(x_test, y_test)[1])
print("Model 3 accuracy after poisoning:", model3.evaluate(x_test, y_test)[1])
```

- Targeted Label Flipping

```
# Create three instances of the model, one for each client
model1 = create_model()
model2 = create_model()
model3 = create_model()

# Train each model on its respective data
model1.fit(x1_train, y1_train, epochs=10, validation_data=(x1_test, y1_test))
model2.fit(x2_train, y2_train, epochs=10, validation_data=(x2_test, y2_test))
model3.fit(x3_train, y3_train, epochs=10, validation_data=(x3_test, y3_test))

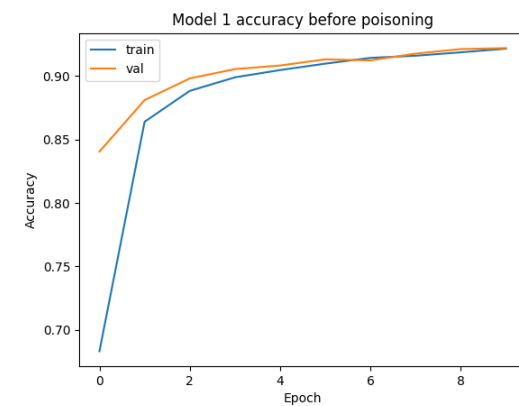
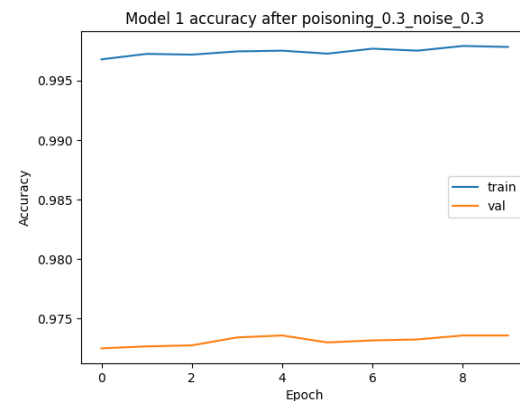
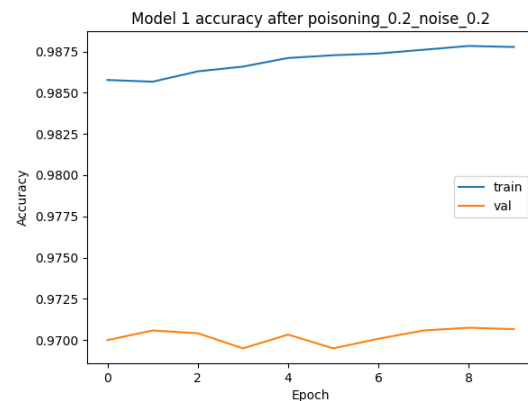
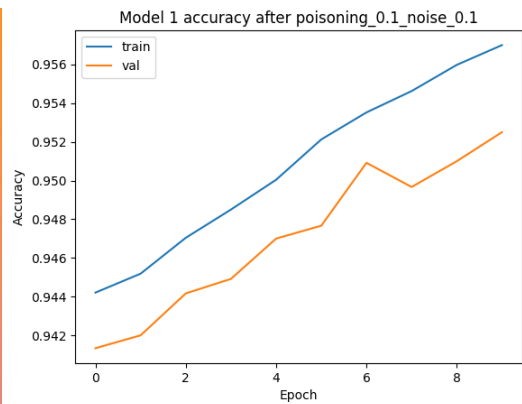
# Evaluate the accuracy of the models on the test data
print("Model 1 accuracy:", model1.evaluate(x_test, y_test)[1])
print("Model 2 accuracy:", model2.evaluate(x_test, y_test)[1])
print("Model 3 accuracy:", model3.evaluate(x_test, y_test)[1])

# Poison the second and third clients randomly training data by flipping the labels of a fraction of the data
poison_fraction = 0.2
poison_indices = np.random.choice(len(y2_train), int(len(y2_train) * poison_fraction), replace=False)
y2_train[poison_indices] = (y2_train[poison_indices] + 1) % 10
poison_indices = np.random.choice(len(y3_train), int(len(y3_train) * poison_fraction), replace=False)
y3_train[poison_indices] = (y3_train[poison_indices] + 1) % 10

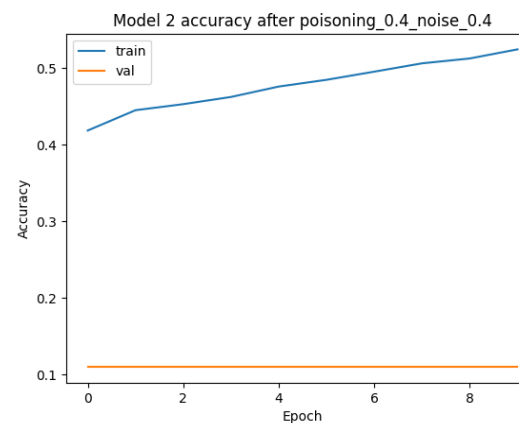
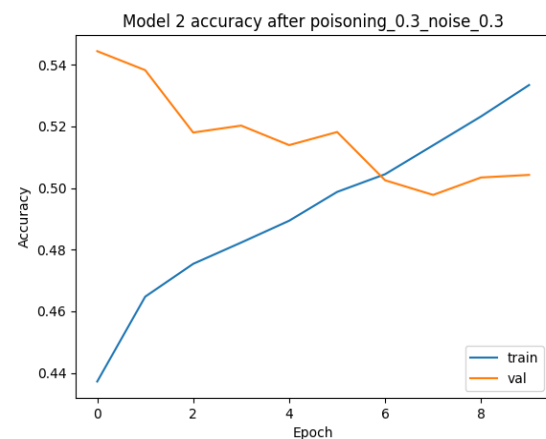
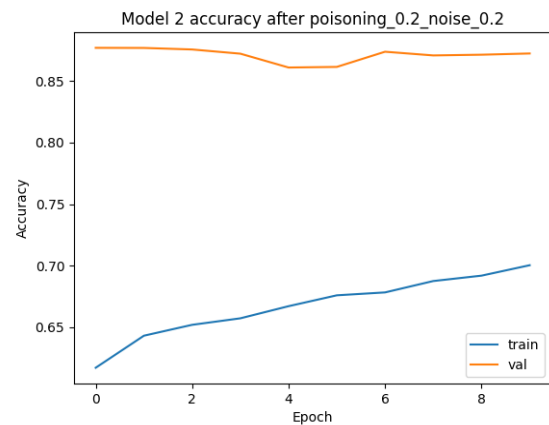
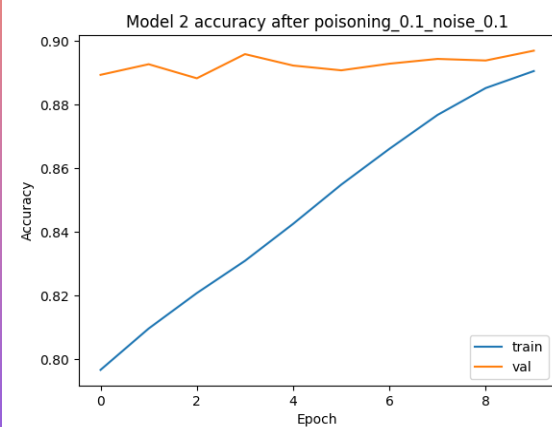
# Retrain the models on the poisoned data
model1.fit(x1_train, y1_train, epochs=10, validation_data=(x1_test, y1_test))
model2.fit(x2_train, y2_train, epochs=10, validation_data=(x2_test, y2_test))
model3.fit(x3_train, y3_train, epochs=10, validation_data=(x3_test, y3_test))

# Evaluate the accuracy of the models on the test data again
print("Model 1 accuracy after poisoning:", model1.evaluate(x_test, y_test)[1])
print("Model 2 accuracy after poisoning:", model2.evaluate(x_test, y_test)[1])
print("Model 3 accuracy after poisoning:", model3.evaluate(x_test, y_test)[1])
```

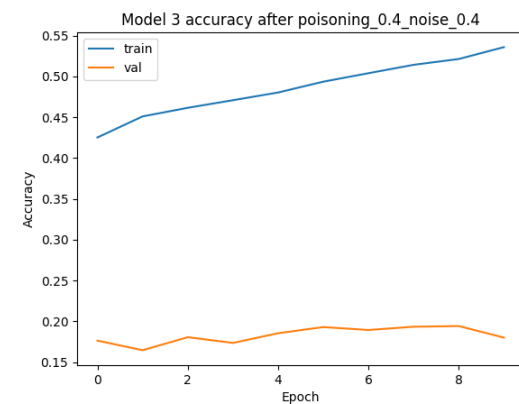
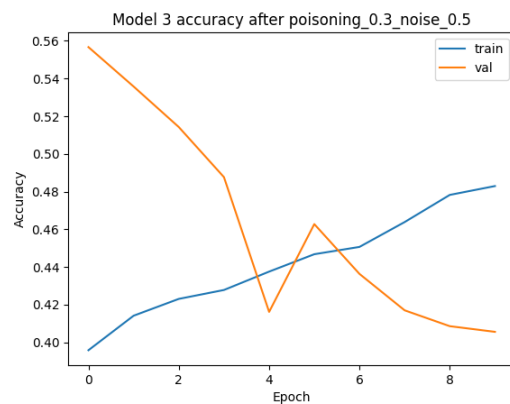
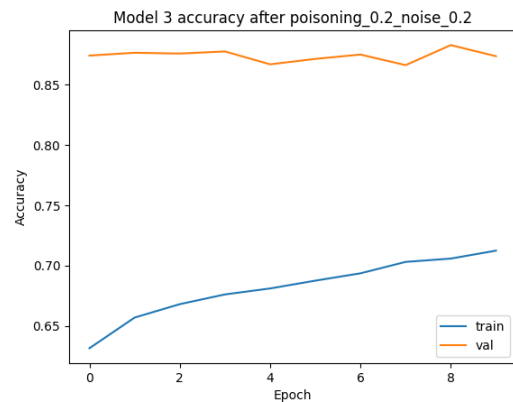
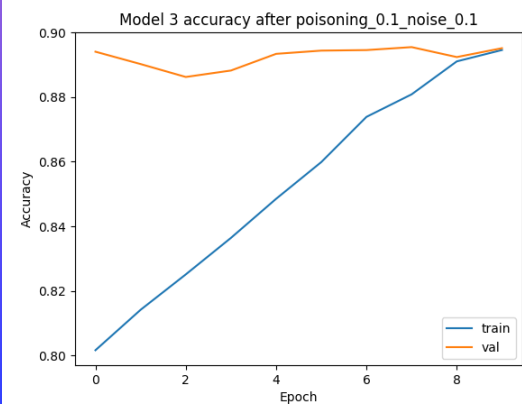
Model 1



Model 2



Model 3



Experimental Setup

- Training Phase 1:

- 50 clients per round
- 100 rounds
- Noise Multiplier - [0.0 - 1.1]

- Training Phase 2:

- 120 clients per round
- 100 rounds
- Noise Multipliers [0.75-1.2]

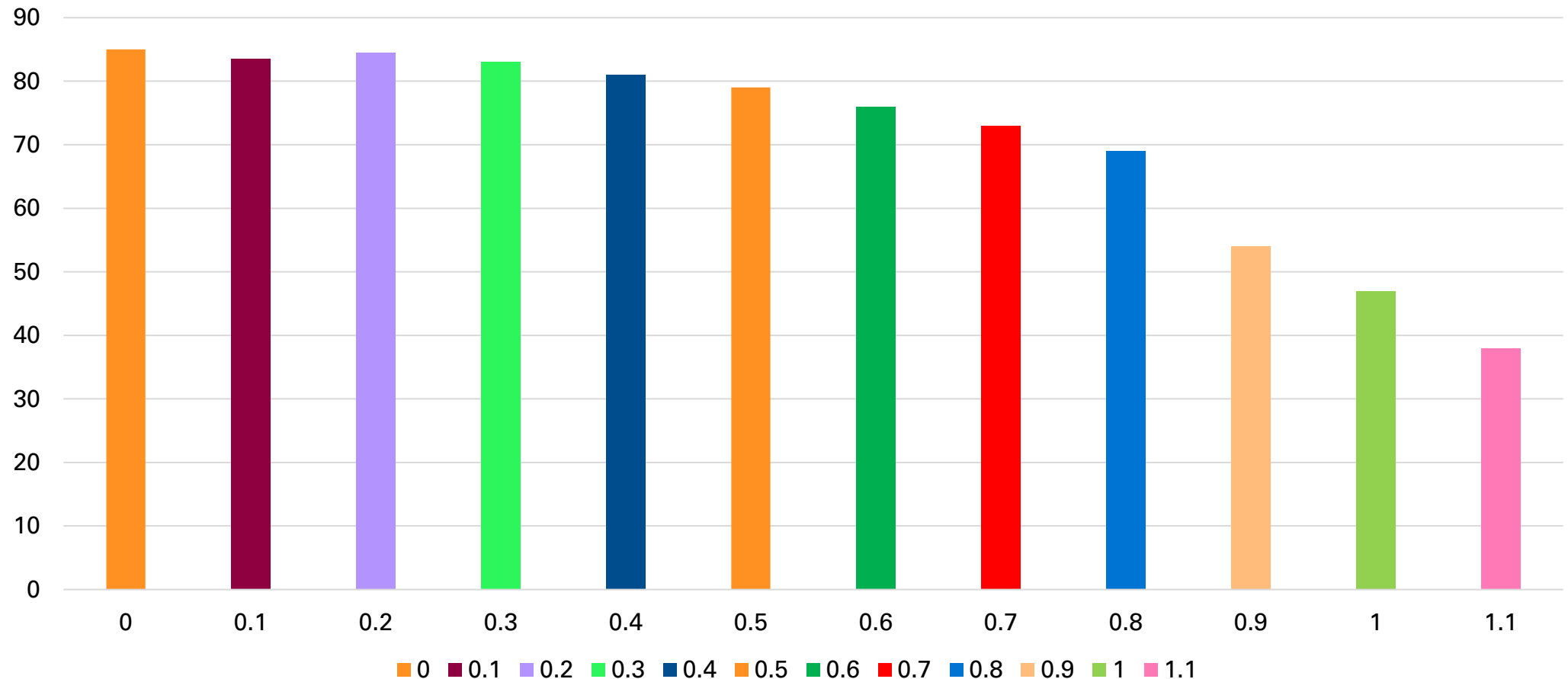
```
24 # Perform search for smallest clients_per_round achieving the target privacy.
25 ✓ clients_per_round = dp_accounting.calibrate_dp_mechanism(
26     make_fresh_accountant, make_event_from_param, target_eps, target_delta,
27     bracket_interval, discrete=True
28 )
29
30 noise_multiplier = clients_per_round * noise_to_clients_ratio
31 ✓ print(f'To get ({target_eps}, {target_delta})-DP, use {clients_per_round} '
32       | f'clients with noise multiplier {noise_multiplier}.')
```

To get (2, 1e-05)-DP, use 120 clients with noise multiplier 1.2.

Results – Train Phase 1

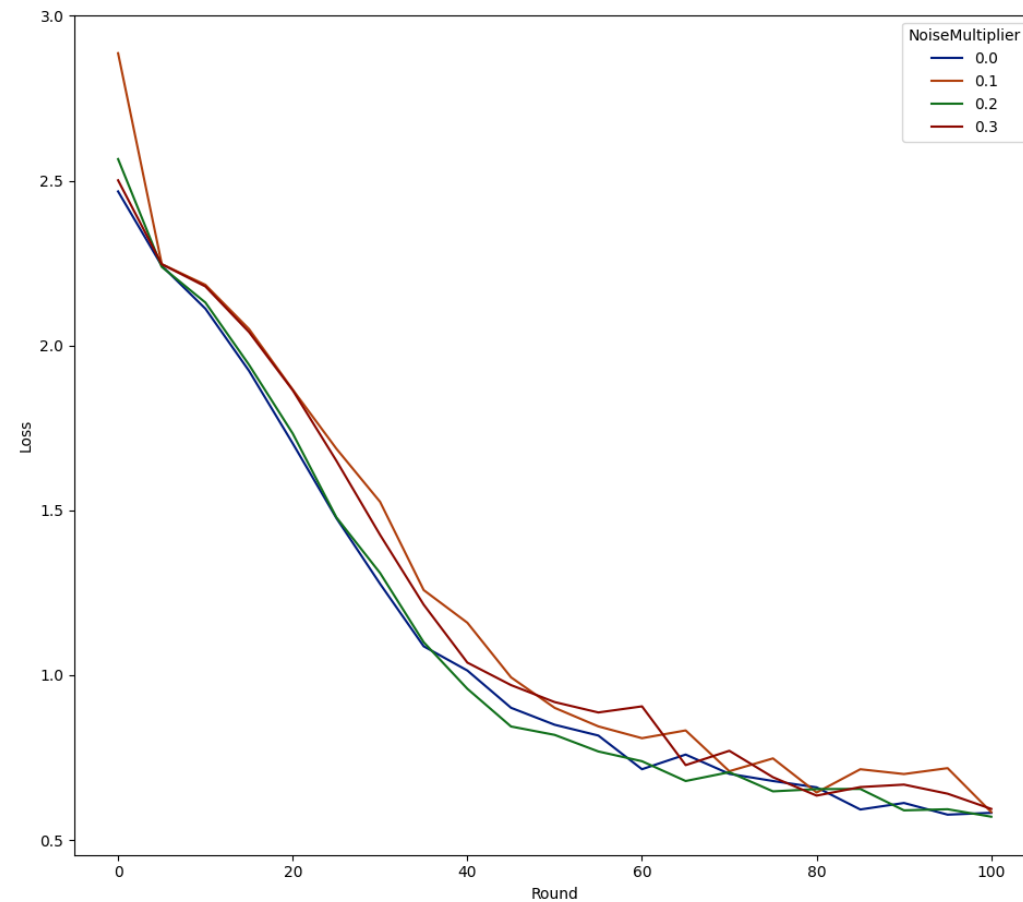
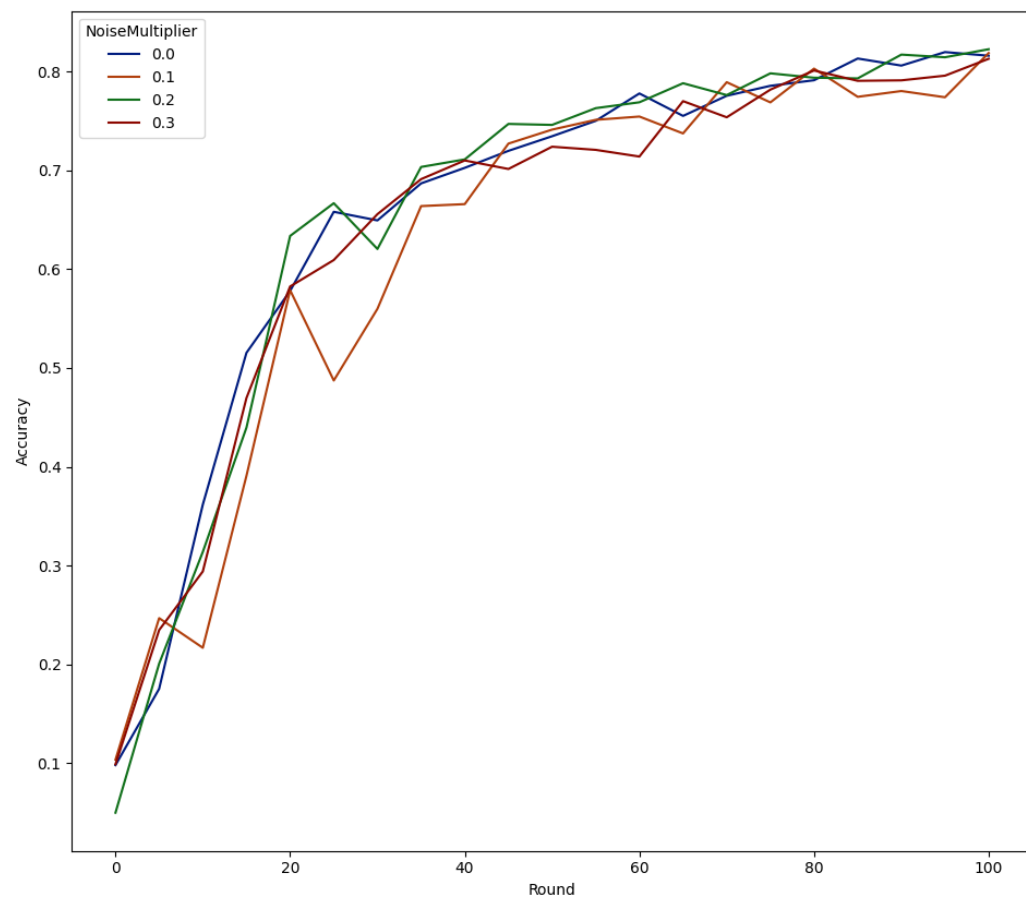
50 clients per rounds, noise - [0.0,1.1]

Final Accuracy vs Noise



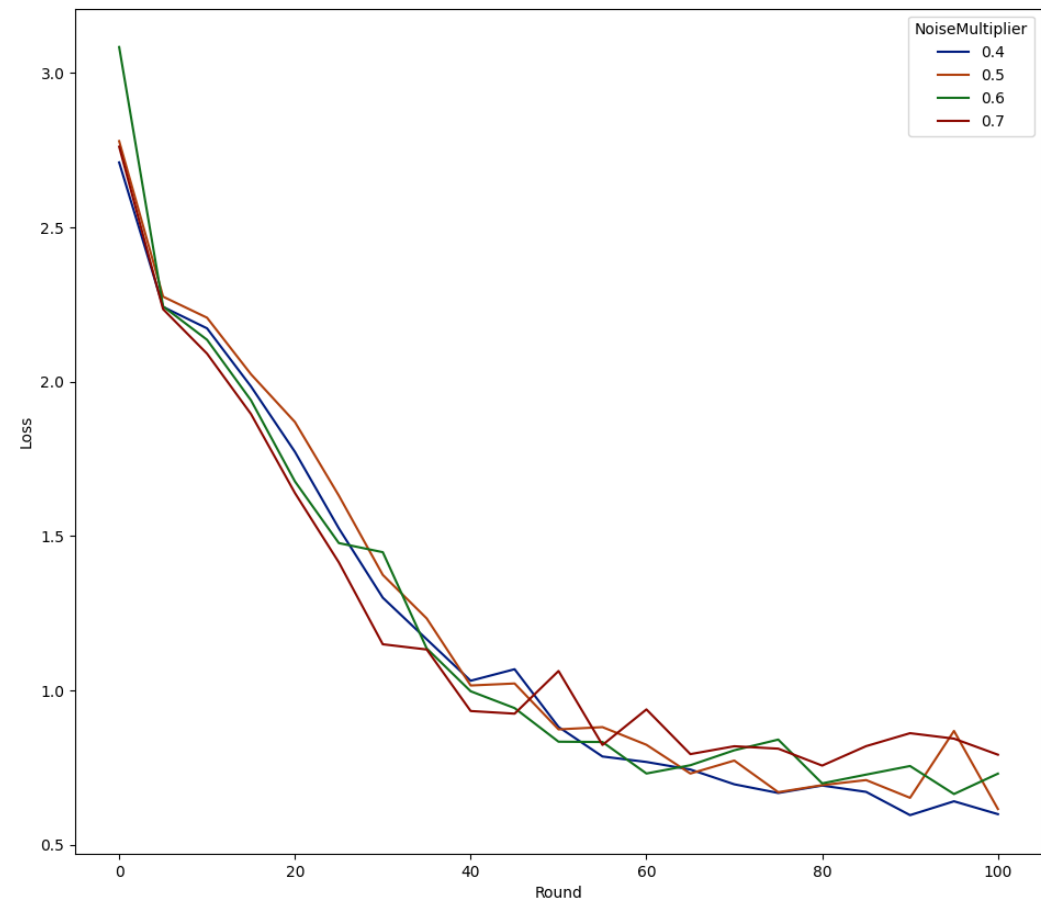
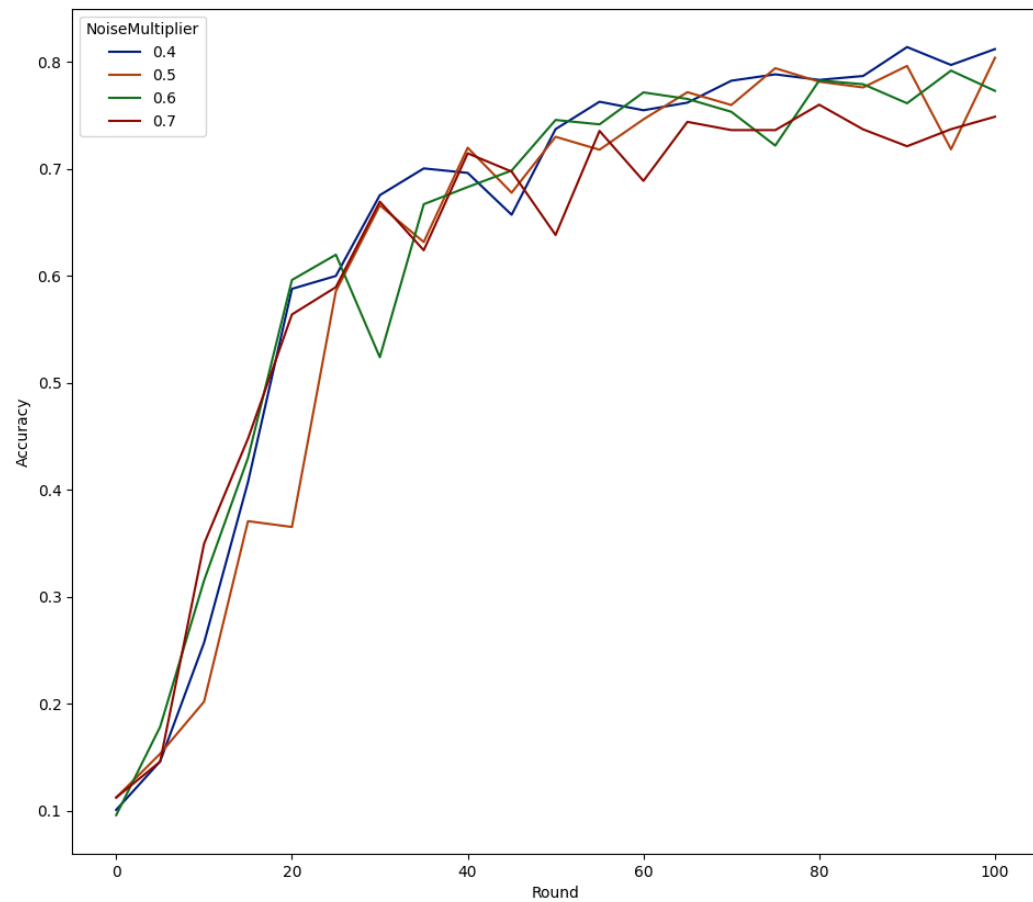
Plots

Noise Multipliers [0-0.3]



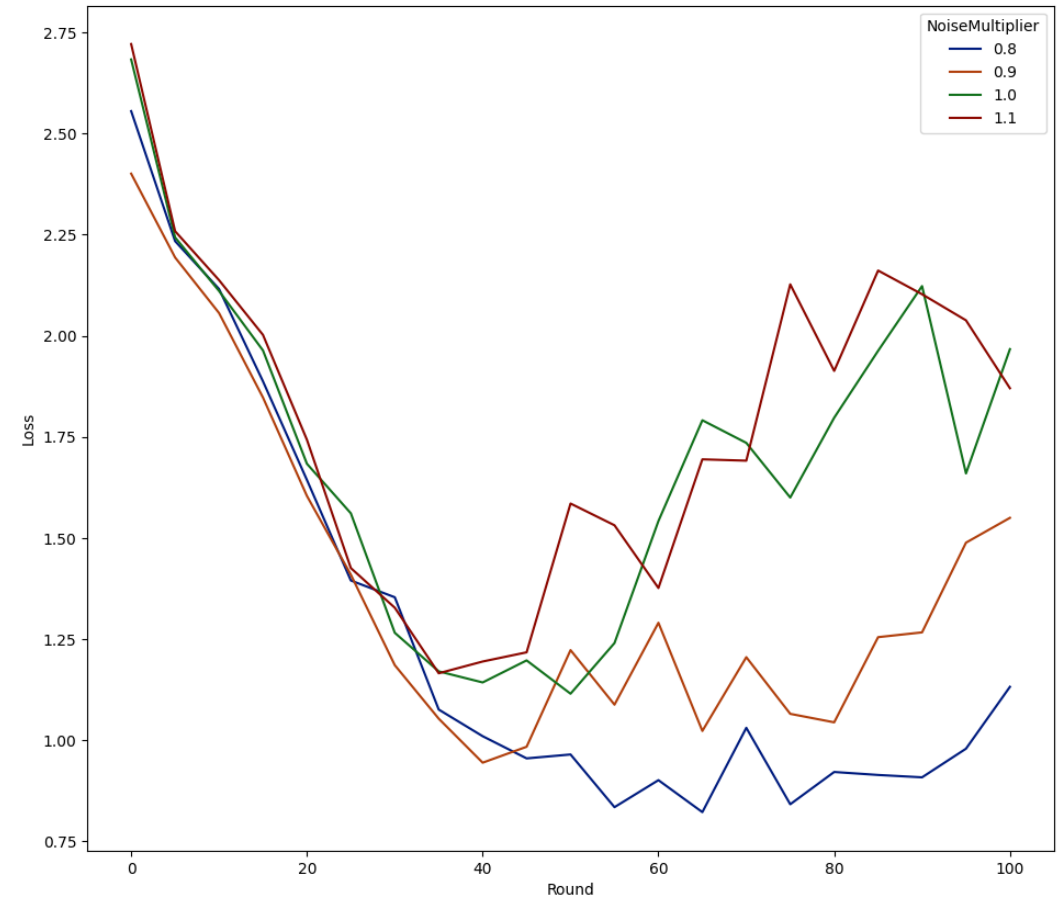
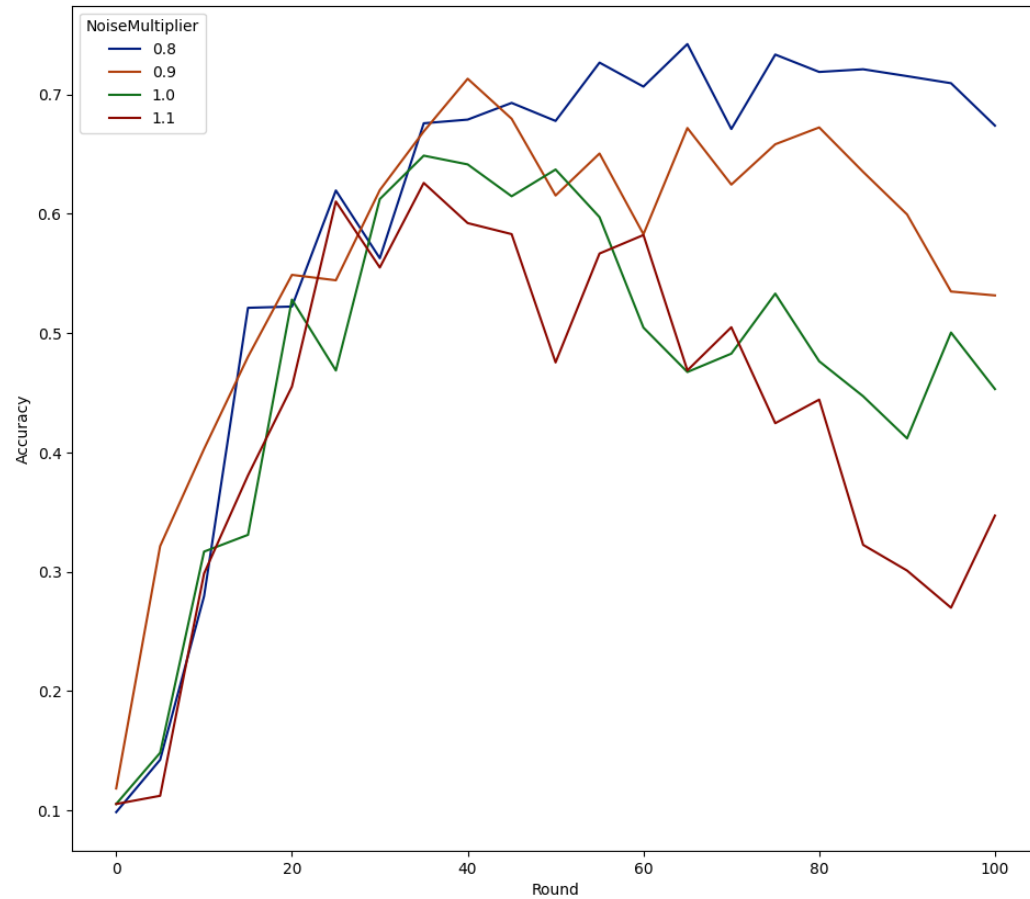
Plots

Noise Multipliers [0.4-0.7]



Plots

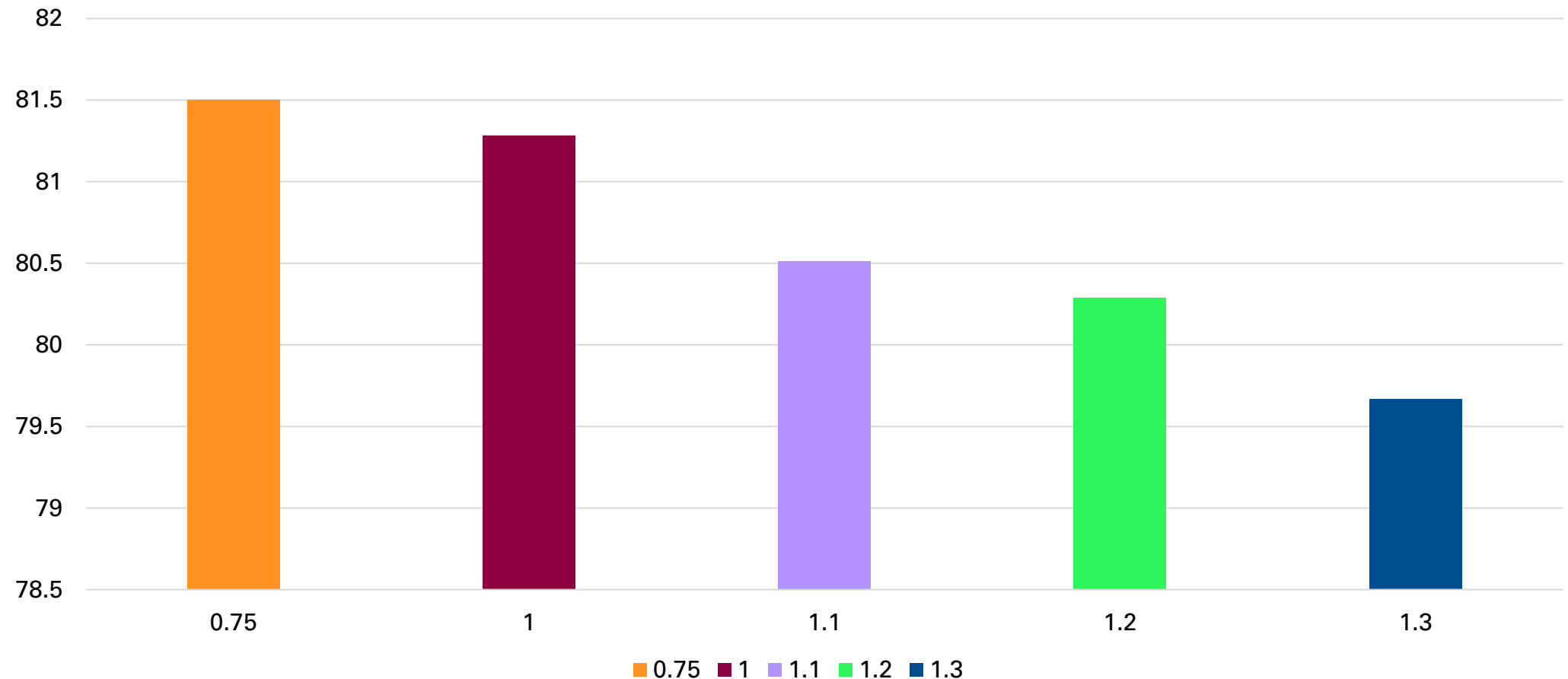
Noise Multipliers [0.8-1.1]



Results – Train Phase 2

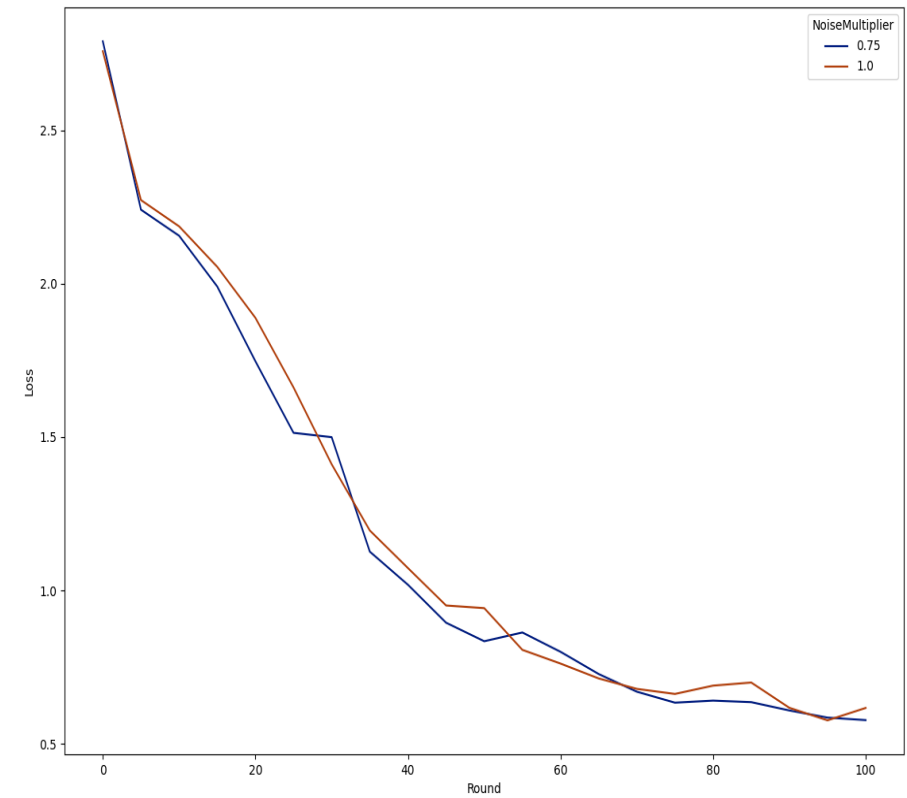
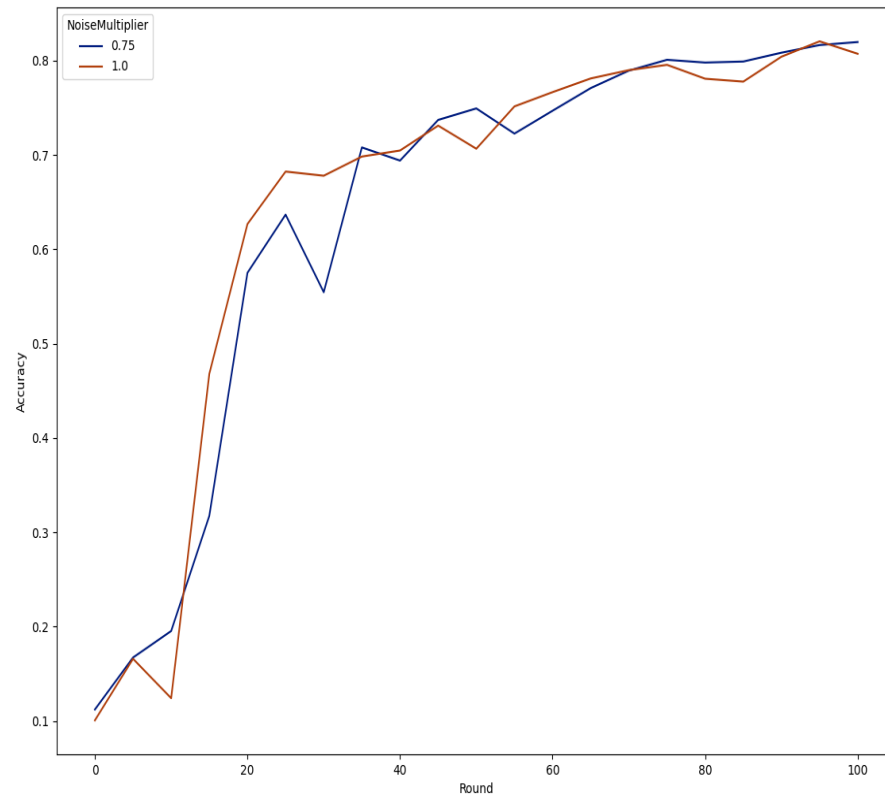
120 Clients per round, noise - [0.75 - 1.3]

Final Accuracy vs Noise



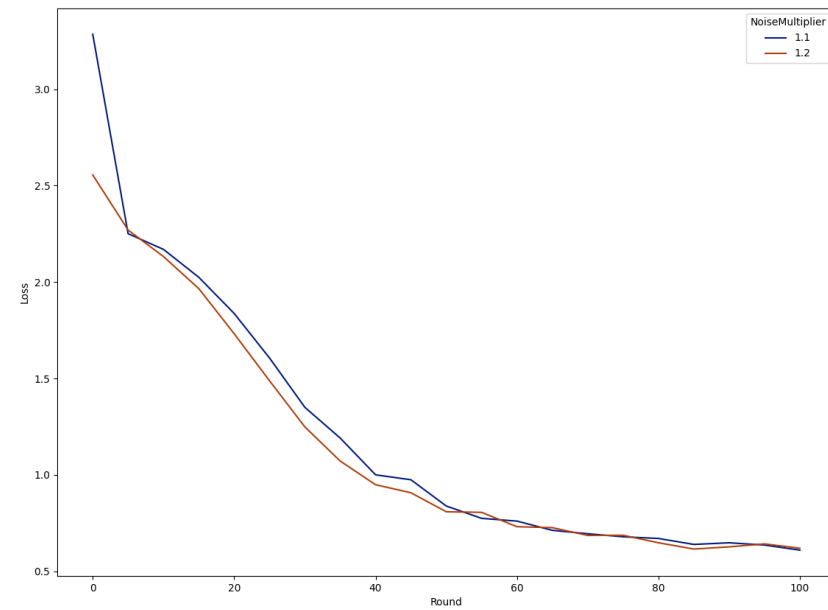
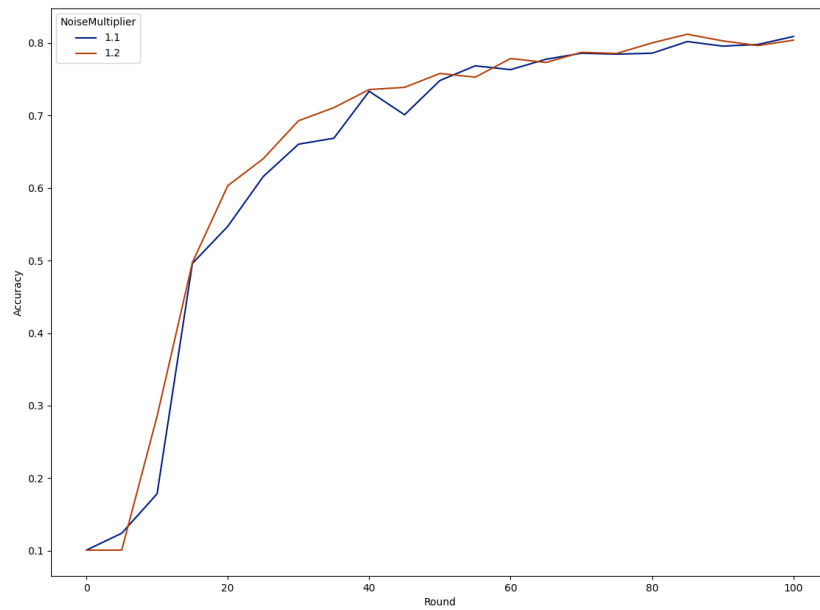
Plots

Noise Multipliers [0.75, 1.0]



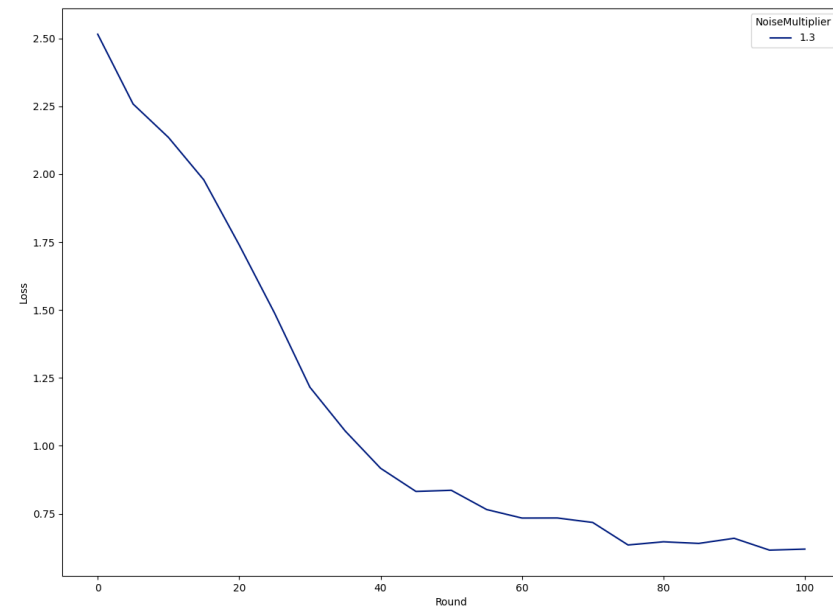
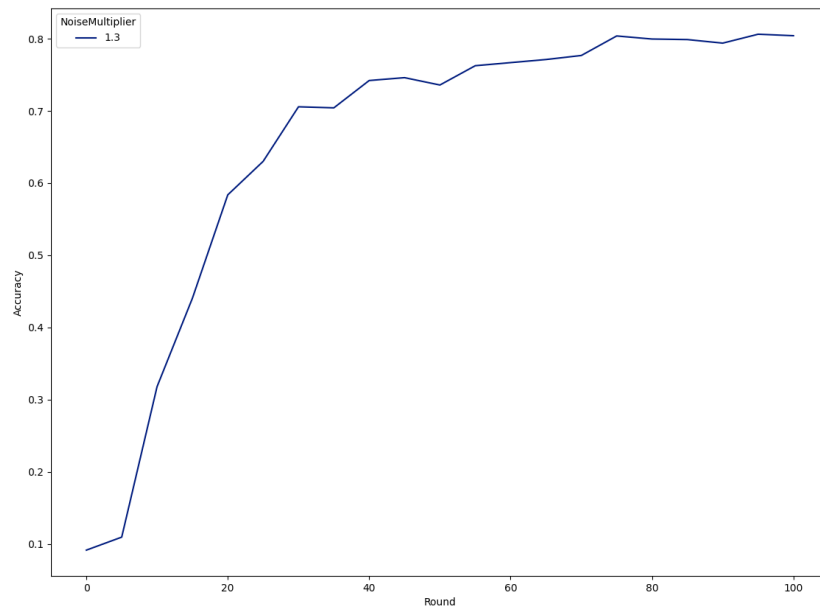
Plots

Noise Multipliers [1.1, 1.2]



Plots

Noise Multipliers [1.3]



Conclusion

- Summary of the Project
- Overview of Privacy-Preserving Techniques in Federated Learning Models
- Implementation of Label Flipping Attacks and Differential Privacy Techniques
- Analysis of Model Accuracy with Noise Variations
- Future Work: Potential Approaches to Improve Privacy and Efficiency



Future Work

1. Novel Attack Strategies
2. Robust Privacy-Preserving Techniques
3. Analysis of Different Federated Learning Settings on Privacy and Security
4. Techniques to Optimize Privacy and Model Accuracy Simultaneously
5. Real-World Impact of Privacy Attacks on Federated Learning Systems

References

- Privacy-Preserving Aggregation in Federated Learning: A Survey Ziyao Liu, Jiale Guo, Wenzhuo Yang, Jiani Fan, Kwok-Yan Lam, Senior Member, IEEE, and Jun Zhao, Member, IEEE
- Defending against Label-Flipping Attacks in Federated Learning Systems with UMAP Deepak Upreti Kongju National University Hyunil Kim Kongju National University Eunmok Yang Kookmin University Changho Seo , Kongju National University
- Dataset Security for Machine Learning: Data Poisoning, Backdoor Attacks, and Defenses
- <https://ieeexplore.ieee.org/document/9308910>
- https://www.tensorflow.org/federated/tutorials/federated_learning_for_image_classification
- https://www.tensorflow.org/federated/tutorials/random_noise_generation

+

o

.

THANK YOU

